Diseño de Compiladores I – Compiladores e Intérpretes
Trabajo Práctico Nº 3 - 2026

La entrega se hará en forma conjunta con el trabajo práctico Nro. 4 (**Fecha a definir**)
OBJETIVO: Se deben incorporar al compilador, las siguientes funcionalidades:

Generación de código intermedio

- Generación de código intermedio para las sentencias ejecutables. Es requisito para la aprobación del trabajo, que el código intermedio sea almacenado en una estructura dinámica.
  En la siguiente tabla se presenta la representación que debe generar cada grupo:

Árbol Sintáctico	*Tercetos*	*Polaca Inversa*
2	4	1
5	7	3
6	11	9
8	15	12
10	17	13
14	19	16
23	20	18
25	21	22
27	26	24
29	36	28
30	37	31
32	38	33
34	39	35
42	41	40
44	43	
	45	

Se deberá generar código intermedio para todas las sentencias ejecutables, incluyendo:

- Asignaciones, Selecciones, Sentencias de control asignadas al grupo, Sentencias **pout**, sentencias **ret** y toda otra sentencia ejecutable que admita el lenguaje**.**
- Para las funciones, se deberá generar el código intermedio correspondiente a las sentencias ejecutables del cuerpo, en forma independiente para cada función. Se podrán generar estructuras independientes para cada una, o ubicarlas todas en la misma estructura que el programa principal, con algún delimitador que permita determinar claramente el inicio y fin de cada una.

Incorporación de información Semántica y chequeos

- ***Incorporación de información a la Tabla de Símbolos:***
  - ***Tipo:***
    - Durante el Análisis Léxico debe quedar registrado, en la Tabla de Símbolos, el tipo de las constantes.
    - Para el resto de los Identificadores, se deberá registrar el tipo en la Tabla de Símbolos a partir de las sentencias declarativas.
  - ***Uso:***
    - Incorporar un atributo **Uso** en la Tabla de Símbolos, indicando el uso de cada identificador en el programa (nombre de variable, nombre de función, nombre de programa, nombre de parámetro, nombre de clase, etc.).
  - ***Otros Atributos:***
    - Se podrán Incorporar atributos adicionales a las entradas de la Tabla de Símbolos, de acuerdo a los temas particulares asignados

Asociación de cada variable con el ámbito al que pertenece:

Para identificar variables, funciones, clases, etc. con el ámbito al que pertenecen, se utilizará "***name mangling***". Es decir, el nombre del símbolo llevará, a continuación de su nombre original, la identificación del ámbito al que pertenece.

Ejemplos:

Ámbitos

```
Main                                                    // main identifica el ámbito global
      // Declaraciones en el ámbito global
      LONGINT a;                                        // la variable a se llamará a:main
      ...
      INTEGER FUNCTION aa(LONGINT x)                    // Ámbito aa
                                                        // La función aa se llamará aa:main
                                                        // El parámetro x se llamará  x:main:aa

            // Declaraciones en el ámbito aa
            INTEGER a;                                  // la variable a se llamará a:main:aa
            ...
            INTEGER FUNCTION aaa(INTEGER w)             // Ámbito aaa
                  // Declaraciones en el ámbito aaa
                  LONGINT x;                            // la variable x se llamará x:main:aa:aaa
                  ...
            begin
                  // Sentencias ejecutables del ámbito aaa
                  ...
            end                                         // Fin Ámbito aaa
      begin
            // Sentencias ejecutables del ámbito aa
            ...
      End;                                              // Fin Ámbito aa

      INTEGER FUNCTION bb(INTEGER z)                    // Ámbito bb
            // Declaraciones en el ámbito bb
            INTEGER a;                                  // la variable a se llamará a:main:bb
               ...
      begin
            // Sentencias ejecutables del ámbito bb
            ...
      End;                                              // Fin Ámbito bb
      ...
      INTEGER FUNCTION cc(INTEGER y)                    // Ámbito cc
            // Declaraciones en el ámbito cc
            INTEGER a;                                  // la variable a se llamará a:main:cc
            ...
      begin
            // Sentencias ejecutables del ámbito cc
            ...
      End;                                              // Fin Ámbito cc
      ...
begin
      // Sentencias ejecutables del ámbito main
      ...
End;                                                    // Fin Ámbito main
```

Chequeos Semánticos:

En esta etapa, se deberán efectuar los siguientes chequeos semánticos, informando errores cuando corresponda:

➢ Se deberán detectar, informando como error:

- Variables no declaradas o no visibles (según reglas de alcance del lenguaje).
- Variables redeclaradas (según reglas de alcance del lenguaje).
- Funciones no declaradas o no visibles (según reglas de alcance del lenguaje).
- Funciones re declaradas (según reglas de alcance del lenguaje).
- Clases no declaradas o no visibles (según reglas de alcance del lenguaje).
- Clases redeclaradas (según reglas de alcance del lenguaje).
- Métodos no declarados o no visibles (según reglas de alcance del lenguaje).
- Métodos redeclarados (según reglas de alcance del lenguaje).
- Uso incorrecto de un símbolo en un contexto determinado. Por ejemplo: intentar usar el nombre de una función como variable en el lado izquierdo de una asignación.
- Toda otra situación que no cumpla con las siguientes reglas:

➢ Reglas de alcance:

- Cada variable, función, clase, método será visible dentro del ámbito en el que fue declarada/o y por los ámbitos contenidos en el ámbito de la declaración.
- Cada variable, función, clase, método será visible a partir de su declaración, con la restricción indicada en el ítem anterior.
- Se permiten variables con el mismo nombre, siempre que sean declaradas en diferentes ámbitos.
- Se permiten funciones con el mismo nombre, siempre que sean declarados en diferentes ámbitos.
- Se permiten clases con el mismo nombre, siempre que sean declaradas en diferentes ámbitos.
- No se permiten variables, funciones, clases con el mismo nombre dentro de un mismo ámbito.
- Otras reglas que correspondan a temas particulares.

➢ Chequeo de compatibilidad de tipos y Conversiones:

- **Tema 33: Conversiones Explícitas de Entero a Punto Flotante**

  Sólo se permitirán operaciones (expresiones aritméticas o comparaciones) con operandos de distinto tipo, si en el código se incluye la conversión explícita de entero a punto flotante **tosf** o **todf** según tema asignado al grupo. Para asignaciones, sólo se permitirá asignar una expresión entera a una variable de punto flotante, si se incluye la conversión correspondiente del lado derecho de la asignación.

  Ejemplos válidos:

  - `z := todf(a + b);` // con z de tipo flotante, a y b de tipo entero.
  - `z := tosf(a) + y;` // con z e y de tipo flotante, a de tipo entero.

  Ejemplos con error:

  - `z := z + a + b;` // con z de tipo flotante, a y b de tipo entero.
  - `a := x + y;` // con a de tipo entero, x e y de punto flotante.

  Donde el tipo entero puede ser shortint, ushortint, integer, uinteger, longint o ulongint dependiendo del tema asignado al grupo; y las variables de punto flotante pueden ser singlef o doubléf según tema asignado

- **Tema 34: Conversiones Explícitas de Punto Flotante a Entero**

  Sólo se permitirán operaciones (expresiones aritméticas o comparaciones) con operandos de distinto tipo, si en el código se incluye la conversión explícita de punto flotante a entero (**tos**, **tous**, **toi**, **toui**, **tol** o **toul**) según tema asignado al grupo. Para asignaciones, sólo se permitirá asignar una expresión de punto floatante a una variable de tipo entero, si se incluye la conversión correspondiente del lado derecho de la asignación.

  Ejemplos válidos:

  - `a := toi(x + y);` // con x e y de tipo flotante, y a de tipo entero.
  - `a := toul(x) + z;` // con x y z de tipo flotante, a de tipo entero.

  Ejemplos con error:

  - `a := z + b;` // con z de tipo flotante, a y b de tipo entero.
  - `a := x + y;` // con a de tipo entero, x e y de punto flotante.

  Donde el tipo entero puede ser shortint, ushortint, integer, uinteger, longint o ulongint dependiendo del tema asignado al grupo; y las variables de punto flotante pueden ser singlef o doubléf según tema asignado

- **Tema 35: Conversiones Implícitas**

  El compilador debe incorporar las conversiones en forma implícita, cuando se intente operar entre operandos de diferentes tipos (uno entero y otro flotante). En todos los casos, la conversión a incorporar será del tipo entero (temas 1-2-3-4-5-6) al tipo de punto flotante asignado al grupo. Por ejemplo, el compilador incorporará una conversión del tipo **integer** a **doublef**, si se intenta efectuar una operación (expresión aritmética o comparación) entre dos operandos de dichos tipos.

  En el caso de asignaciones, si el lado izquierdo es de uno de tipo entero (1-2-3-4-5-6), y la expresión del lado derecho es de punto flotante, se deberá informar error por incompatibilidad de tipos. En cambio, si el lado izquierdo es del tipo de punto flotante asignado al grupo, y el lado derecho es de tipo entero (1-2-3-4-5-6), el compilador deberá incorporar la conversión que corresponda al lado derecho de la asignación.

  Ejemplo válido:

  - `z := a + b;` // con z de tipo flotante, a y b de tipo entero.
    // El compilador deberá incorporar la conversión de entero a flotante que corresponda.

  Ejemplo con error:

  - `a := x + y;` // con a de tipo entero, x e y de punto flotante.

  Donde el tipo entero puede ser shortint, ushortint, integer, uinteger, longint o ulongint dependiendo del tema asignado al grupo; y las variables de punto flotante pueden ser singlef o doubléf según tema asignado

NOTA:

- Para **Tercetos** y **Árbol Sintáctico**, los chequeos de tipos y la incorporación de conversiones implícitas se deben efectuar durante la generación de código intermedio
- Para **Polaca Invers**a, el chequeo de compatibilidad de tipos y la incorporación de conversiones implícitas (si corresponde) se debe efectuar en la generación de código Assembler (Trabajo Práctico 4).

➢ Semántica Asociada a Funciones y Métodos

Registro de información en la Tabla de Símbolos

En esta etapa, se deberá registrar:

- Información referida a la función o método:
  - El tipo devuelto
  - El tipo y nombre de los parámetros.
- Otros atributos que el compilador requiera para permitir esta funcionalidad en el lenguaje.

Código intermedio

- Se deberá generar código para cada función y/o método declarado. El código de cada función o método se podrá generar en una única estructura, junto con el programa principal, o bien, utilizando una estructura independiente para el programa principal y para cada función.

Invocaciones

Cuando se detecte una **invocación**, se deberá generar código para la misma, efectuando los siguientes chequeos:

- El número de los parámetros reales al invocar una función o método debe coincidir con el número de parámetros formales en la declaración correspondiente.
- El pasaje de parámetros será por copia. Por lo tanto, previo a la invocación, se deberá efectuar la copia de cada parámetro real sobre el parámetro formal correspondiente, en el orden en que aparecen, o en el orden indicado en forma explícita para los temas 19 y 20
- El tipo de cada parámetro real debe ser compatible con el tipo del parámetro formal correspondiente. Para chequear y/o lograr dicha compatibilidad, se aplicarán las reglas detalladas para los temas 33, 34 y 35 según corresponda al grupo.

Retornos

- Las funciones pueden o no tener retorno (excepto el caso particular indicado en el tema 21.
- Para aquellas funciones que no tengan retorno, se asumirá un valor de retorno por defecto, del tipo devuelto por la función.

-

Semántica de los temas particulares asignados.

➢ Temas 11 a 14: Sentencias de Control Iterativas

- **Tema 11**: ***while repeat***

```
while ( <condicion> ) repeat <bloque_de_sentencias_ejecutables> ;
```

El bloque de sentencias ejecutables se ejecutará mientras la condición sea verdadera.

- **Tema 12**: ***repeat until***

```
repeat <bloque_de_sentencias_ejecutables> until ( <condicion> );
```

El bloque de sentencias ejecutables se ejecutará hasta que la condición sea verdadera. La evaluación de la condición se efectuará luego de la ejecución del bloque.

- **Tema 13**: ***repeat while***

```
repeat <bloque_de_sentencias_ejecutables> while ( <condicion> );
```

El bloque de sentencias ejecutables se ejecutará mientras la condición sea verdadera. La evaluación de la condición se efectuará luego de la ejecución del bloque.

- **Tema 14**: ***from to***

```
from  ID := CTE1 to CTE2 by CTE3 repeat < bloque_sentencias_ejecutables >;
```

La variable de control indicada por ID, comenzará la iteración con el valor inicial, y se actualizará, al final de cada iteración, con el valor de actualización. El bloque de sentencias se ejecutará mientras el valor de la variable de control no alcance el valor final

Si CTE1 es menor o igual que CTE2:
En cada ciclo:
- La condición controlará que la variable de control sea menor que CTE2.
- El bloque de sentencias se ejecutará mientras dicha condición sea verdadera.
- La variable de control se incrementará en un valor igual a CTE3 al finalizar la ejecución del bloque de sentencias.

Si CTE1 es mayor que CTE2.
En cada ciclo:
- La condición controlará que la variable de control sea mayor que CTE2.
- El bloque de sentencias se ejecutará mientras dicha condición sea verdadera.
- La variable de control se decrementará en un valor igual a CTE3 al finalizar la ejecución del bloque de sentencias.

Ejemplos:

```
from i:= 1$i to 10$i by 1$i repeat begin … end;
```

// La ejecución del bloque se iniciará con un valor inicial 1 para la variable i, y se ejecutará mientras su valor sea menor que 10. En cada ciclo se incrementará el valor de i en 1.

```
from j:= 10$l to 0$l by 2$ repeat begin … end;
```

// La ejecución del bloque se iniciará con un valor inicial 10 para la variable j, y se ejecutará mientras su valor sea mayor que 0. En cada ciclo se decrementará el valor de j en 2.

Se deberán efectuar los siguientes chequeos de tipo:
- ID debe ser una variable de tipo entero (1-2-3-4-5-6).
- CTE1, CTE2 y CTE3 deben ser constantes de tipo entero (1-2-3-4-5-6).

➢ Temas 17 y 18: Asignaciones en expresiones

- **Tema 17**: ***Asignaciones de expresiones en expresiones***

  Se debe generar código intermedio para las asignaciones indicadas con el operador '=', con la misma estructura que las asignaciones comunes.

  Se deben verificar los tipos (considerando conversiones si corresponde) para estas asignaciones con las mismas reglas que en las asignaciones normales.

- **Tema 18**: ***Asignaciones de identificadores o constantes en expresiones***

  Se debe generar código intermedio para las asignaciones indicadas con el operador '=', con la misma estructura que las asignaciones comunes.

  Se deben verificar los tipos (considerando conversiones si corresponde) para estas asignaciones con las mismas reglas que en las asignaciones normales.

➢ Temas 19 a 21: Funciones con invocaciones y retornos con características especiales

- **Tema 19**: ***Orden de evaluación de parámetros reales obligatorio (con asignaciones en expresiones)***

  Se deben efectuar los chequeos y consideraciones indicados para las invocaciones en general.

  Se debe chequear que el número de valores constantes que indican el orden de evaluación de los parámetros coincida con el número de parámetros de la función.

  Se debe generar código para la copia de los parámetros reales a los formales en el orden indicado en esa lista de valores constantes.

  Por ejemplo, dada la definición de función:

```
integer function f(integer p1, integer p2, integer p3)
begin
      ret (p1+p2+p3);
end;
```

  Para los siguientes ejemplos de invocaciones

```
a := 1$i;
b := 2$i;
```

  Si se invoca:

```
c := f(a,b=4,a+b) [1,2,3]     // la función será invocada con los valores 1, 4 y 5, retornando el valor 10
```

  Si se invoca:

```
c := f(a,b=4,a+b) [1,3,2]     // la función será invocada con los valores 1, 4 y 3, retornando el valor 8
```

- **Tema 20**: ***Orden de evaluación de parámetros reales opcional (con asignaciones en expresiones)***

  Se deben efectuar los chequeos y consideraciones indicados para las invocaciones en general.

  Se debe chequear que el número de valores constantes que indican el orden de evaluación de los parámetros coincida con el número de parámetros de la función.

  Se debe generar código para la copia de los parámetros reales a los formales.

  - Si existe indicación de orden de evaluación, la copia debe efectuarse en dicho orden. en el orden indicado en esa lista de valores constantes.
  - Si no se indica orden, la copia debe efectuarse en el orden de la lista de parámetros

  Por ejemplo, dada la definición de función:

```
integer function f(integer p1, integer p2, integer p3)
begin
      ret (p1+p2+p3);
end;
```

Para los siguientes ejemplos de invocaciones

```
a := 1$i;
b := 2$i;
```

Si se invoca:

```
c := f(a,b=4,a+b)          // la función será invocada con los valores 1, 4 y 5, retornando el valor 10
```

Si se invoca:

```
c := f(a,b=4,a+b) [1,3,2]  // la función será invocada con los valores 1, 4 y 3, retornando el valor 8
```

- **Tema 21**: ***Funciones con retornos de tipos diferetnes***

  Para funciones declaradas como AUTO se deben efectuar los chequeos y consideraciones indicados para las invocaciones en general, considerando de manera diferente lo referido a retornos.

  Estas funciones deben tener al menos un retorno, pero permiten también varios retornos de tipos diferentes. El compilador deberá determinar, para este tipo de funciones, el tipo de retorno más abarcativo cuando existen retornos de tipos deiferentes. De este modo, una invocación retornará dicho tipo, que será considerado para los chequeos correspondientes en el contexto donde aparezca dicha invocación.

➢ Temas 22 y 23: Declaraciones especiales

- **Tema 22**: ***Variables en tiempo de compilación***

  El compilador deberá registrar, para estas variables, el valor que reciban a lo largo del programa. Cada vez que la variable sea usada en código, en el código intermedio se utilizará, en lugar del nombre de esa variable, el valor registrado en ese momento.

  El compilador debe verificar que estas variables se usen en contextos donde se pueda determinar su valor en tiempo de compilación.

Ejemplos válidos				

comptime integer x;
integer z;
x := 3$i;
x := x + 1$i;
z := x;
	comptime integer x;
	integer z;
	x := 3$i;
	if (x>1$i)
	  z := z + 1$i;
	…
		comptime integer x;
		integer z;
		x := 3$i;
		while (x<10$i) begin
		  x := x + 1$i;
		end;
			Integer function
			f(comptime integer x)
			begin
			  pout (x);
			end;
			a := f(3$i);
				Integer function
				f(integer p)
				begin
				  Pout(p);
				End;
				comptime integer x;
				integer z;
				…
				x:=3$i;
				z :=f(x);

Ejemplos inválidos				

comptime integer x;
integer z;
x := 3$i;
x := x + 1$i;
…
x := z;
	comptime integer x;
	integer z;
	…
	x := 3$i;
	while (x<z) begin
	  x := x + 1$i;
	end;
		integer z;
		Integer function
		f(comptime integer
		x) begin
		  pout (x);
		end;
		a:= f(z);		

- **Tema 23**: ***Enumeraciones***

  El compilador deberá:
  - Registrar en la Tabla de Símbolos el tipo definido por el usuario, como tipo.
  - Chequear que el tipo exista cuando se declaran variables del tipo definido por el usuario
  - Chequear que el valor asignado a una variable declarada del tipo enumerado esté dentro de los valores de la enumeración, cuando el valor asignado sea una constante.
    Aclaración: Si el valor asignado es una variable o expresión cuyo valor no se conozca en compilación, el chequeo se postergará a la siguiente etapa.

➢ Temas 24 al 27: Clases/Objetos

- **Tema 24**: ***Asignación por estructura en objetos con código***

  En la declaración de las clases, el programador **debe** incluir, luego del nombre de la clase, un código.

  El compilaodr debe chequear que las clases que tienen el mismo código tengan la misma estructura o sean compatibles según las reglas de compatibilidad del lenguaje.

  Se permitirán asignaciones entre objetos de clases diferentes, sólo si dichas clases poseen el mismo código.

- **Tema 25**: ***Asignación por estructura en objetos sin código***

  Se permitirán asignaciones entre objetos de clases diferentes, sólo si dichas clases poseen la misma estructura o son compatibles según las reglas de compatibililidad del lenguaje.

- **Tema 26**: ***import/export por atributos y métodos***

  En la declaración de las clases se puede incluir, luego de la declaración de un atributo o de un método, una lista de clases a las que ese atributo o método se exporta.

```
class a begin
   integer x1 export to b, c;
   singlef x2 export to c;
   integer m1(integer p) begin
   …
   end; export to b;
end;

class b import from a begin
   integer x3; export to c;
   integer m2(A a1) begin
      a1.x1 ;= 1$i;
      a1.m1(1$i);
   end;
end;

class c import from a, b begin
   integer m3(A a1, B b2) begin
      a1.x2 := 2$i;
      b2.x3 := 3$i;
   end;
end;
```

- Al utilizar un atributo o invocar a un método, el compilador deberá chequear su existencia. Si no lo encuentra en la clase del objeto correspondiente, deberá buscar en las clases de las cuales importa dicha clase, informando error si no se encuentra tampoco en esas clases.
- El compilador deberá registrar, en la Tabla de Símbolos, la información necesaria para dicho chequeo.
-

- **Tema 27**: ***Clases Amigas***

En la declaración de una clase se puede incluir, precediendo a la declaración de un atributo o de un método, la palabra private, y agregar, en el cuerpo de la clase, una lista de clases y funciones amigas, como se indica en el siguiente ejemplo:.

```
CLASS ca
BEGIN
  Private INTEGER a;              // declaración de atributo privado
  Private INTEGER m(integer pa)
  BEGIN                           // declaración de método privado
     …
  END;
  Friend cb;                      // declara a la clase cb como amiga
  Friend f;                       // declara a la función f como amiga
END;
```

- Al utilizar un atributo o invocar a un método, el compilador deberá chequear su existencia.
- Los métodos de estas clases amigas y las funciones amigas van a poder acceder a los atributos y métodos privados de la clase que las declaró como amigas
- El compilador deberá registrar, en la Tabla de Símbolos, la información necesaria para dicho chequeo.

➢ Temas 28 y 29: Acceso a atributos

- **Tema 28**: ***Acceso tradicional a atributos***

El acceso a un atributo de una clase se efectuará como se indicó en la declaración general para sentencias ejecutables. Es decir utilizando el punto.
El compilador deberá chequear su existencia y visibilidad, de acuerdo con las reglas de alcance del lenguaje.

- **Tema 29**: ***Acceso posicional a atributos***

El acceso a un atributo de una clase, **para este tema**, se efectuará con una notación posicional.
El compilador deberá chequear su existencia y visibilidad, de acuerdo con las reglas de alcance del lenguaje.

➢ Temas 30 a 32: Herencia múltiple

Para los 3 temas:
En la declaración de una clase se puede incluir, en el cuerpo de la clase, una o más clases de las cuales esta clase hereda, mediante una sentencia extend seguida de una lista de nombres de clase, como se indica en el ejemplo:

```
class ca
begin
  integer x;
  integer z;
  extends cb, cc, cd;   // la clase ca hereda de cb, cc, cd
end;
```

Tema 30: ***Desambiguado Opción 1***

- Al utilizar un atributo o invocar a un método, el compilador deberá chequear su existencia y visibilidad. Si el atributo o método no está declarado en la clase del objeto, se deberá buscar en las clases de las cuales hereda.
- En caso de existir atributos y/o métodos con el mismo nombre en dichas clases, la búsqueda debe hacerse en el orden de la lista de herencia.
- El compilador deberá registrar, en la Tabla de Símbolos, la información necesaria para dicho chequeo.

Tema 31: ***Desambiguado con prefijado***

- Al utilizar un atributo o invocar a un método, el compilador deberá chequear su existencia y visibilidad.
- Si se pretende utilizar un atributo o método de una clase que se encuentra en la lista de clases heredadas, la misma se debe indicar en forma explícita.
- El compilador deberá registrar, en la Tabla de Símbolos, la información necesaria para dicho chequeo.
- Por ejemplo:

```
ca a1;
a1.x            // acceder al atributo x de la clase ca
a1.cb.x         // acceder al atributo x de la clase cb
a1.cc.x         // acceder al atributo x de la clase cc

o

a1[0$i]         // acceder al primer atributo de la clase ca
a1.cb[1$i]      // acceder al segundo atributo de la clase cb
```

Tema 32: ***Desambiguado Opción 3***

- Al utilizar un atributo o invocar a un método, el compilador deberá chequear su existencia y visibilidad. Si el atributo o método no está declarado en la clase del objeto, se deberá buscar en las clases de las cuales hereda.
- En caso de existir atributos y/o métodos con el mismo nombre en dichas clases, se deberá informar error.
- El compilador deberá registrar, en la Tabla de Símbolos, la información necesaria para dicho chequeo.

➢ Tema adicional para alumnos del plan 2011

Aquellos grupos en los cuales uno o más integrantes sean alumnos del plan 2011, deberán incorporar la optimización Reducción Simple sobre la representación intermedia terminada. Esta optimización se explicará en la próxima clase teórica.

Salida del compilador

1) Código intermedio, de alguna forma que permita visualizarlo claramente. Para Tercetos y Polaca Inversa, mostrar la dirección (número) de cada elemento, de modo que sea posible hacer un seguimiento del código generado. Para Árbol Sintáctico, elegir alguna forma de presentación que permita visualizar la estructura del árbol.
   Ejemplos:

Tercetos:

```
…
20   ( + , a , b )
21   ( / , [20] , 5 )
22   ( := , z , [21])
…
```

Polaca Inversa:

```
…
10 <
11 a
12 b
13 18
14 BF
15 c
16 10
17 :=
…
```

Árbol Sintáctico:

Por ejemplo:

Raíz
Hijo izquierdo
Hijo izquierdo
Hijo derecho
Hijo derecho
Hijo izquierdo
…
Hijo derecho

2) Si bien el código intermedio debe contener referencias a la Tabla de Símbolos, en la visualización del código se deberán mostrar los nombres de los símbolos (identificadores, constantes, cadenas de caracteres) correspondientes.
3) Los errores generados en cada una de las etapas (Análisis Léxico, Sintáctico y durante la Generación de Código Intermedio) se deberán mostrar todos juntos, indicando la descripción de cada error y la línea en la que fue detectado.
4) Contenido de la Tabla de Símbolos.

Nota: No se deberán mostrar las estructuras sintácticas ni los tokens que se pidieron como salida en los trabajos prácticos 1 y 2, a menos que en la devolución de la primera entrega se solicite lo contrario.